


**UNIVERSITÀ DEGLI STUDI DI SALERNO**

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

# TEST INCIDENT REPORT

*Ingegneria del Software: Tecniche Avanzate – Test Incident Report*

| | |
|---|---|
| Mario Celzo | Matricola: NF22500193 |
| Renato Mancino | Matricola: NF22500242 |
| Alfonso Maria Ferrara | Matricola: NF22500210 |

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

# Indice


**1 Introduzione** **1**

1.1 Obiettivi del Documento . . . . . 1

**2 Incident Report** **2**

2.1 Test Incident n.1 . . . . . 2

2.1.1 Descrizione del Problema . . . . . 2

2.1.2 Risoluzione Tecnica . . . . . 2

2.1.3 Esito . . . . . 3

2.2 Test Incident n.2 . . . . . 3

2.2.1 Descrizione del Problema . . . . . 3

2.2.2 Risoluzione Tecnica . . . . . 3

2.2.3 Esito . . . . . 4

2.3 Test Incident n.3 . . . . . 4

2.3.1 Descrizione del Problema . . . . . 4

2.3.2 Risoluzione Tecnica . . . . . 4

2.3.3 Esito . . . . . 5

2.4 Test Incident n.4 . . . . . 5

2.4.1 Descrizione del Problema . . . . . 5

2.4.2 Risoluzione Tecnica . . . . . 6

2.4.3 Esito . . . . . 6

CAPITOLO 1

# Introduzione

Il presente documento costituisce l'Incident Report del progetto BiblioFlow. In questo documento vengono analizzate le problematiche identificate durante l'esecuzione della suite di test che caratterizza la versione di partenza e durante la sua riesecuzione come suite di regressione. Nell'ambito di questa analisi vengono pertanto presi in esame tutti i livelli di testing coinvolti.

## 1.1 Obiettivi del Documento

L'obiettivo primario è fornire una mappatura chiara delle criticità individuate nella suite di test, distinguendo i difetti del sistema da quelli dei test stessi. Questa attività di revisione e correzione è propedeutica al miglioramento della qualità complessiva del sistema e all'aumento dell'affidabilità della suite di test, garantendo una base solida per gli sviluppi futuri.

CAPITOLO 2

# Incident Report

In questa sezione vengono dettagliati gli incidenti specifici affrontati durante la fase di validazione. Per ogni anomalia viene riportata la descrizione dell'errore, l'analisi delle cause e l'intervento correttivo applicato.

## 2.1 Test Incident n.1

**Componente:** Test di Sistema – esecuzione della suite pre-modifica sulla versione di partenza (tag `baseline-pre-cr-bf-01`)

**Severità:** Minore (bloccante per l'esecuzione)

**Data:** 28 agosto 2026

### 2.1.1 Descrizione del Problema

La suite di caratterizzazione non può essere eseguita sul tag che congela la versione di partenza: il tag non contiene né i file di test né lo script `test`. Lanciando `npm test` sul `package.json` del tag (npm 10) si ottiene:

```
$ npm test
npm error Missing script: "test"
```

La versione di partenza non conteneva alcuna infrastruttura di test: niente Vitest, niente script `test`, nessun database di test né fixture. Il tag, che per definizione non può essere modificato, non era quindi eseguibile come riferimento dei test.

### 2.1.2 Risoluzione Tecnica

**Codice Errato:** gli script del `package.json` della versione di partenza non prevedono i test.

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "postinstall": "prisma generate",
  ...
}
```

**Codice Corretto:** invece di modificare il tag, l'infrastruttura di test è stata aggiunta in un commit dedicato (`7469969`), che introduce Vitest, le fixture e gli script; `test:db:prepare` crea un PostgreSQL effimero e rifiuta qualunque indirizzo che non identifichi esplicitamente un database di test.

```
"test": "vitest run --passWithNoTests",
"test:db:prepare": "tsx scripts/test-database.ts prepare",
"test:db:down": "tsx scripts/test-database.ts down",
```

Il commit non tocca `src` né `prisma`. Rispetto al tag, nelle due cartelle cambiano soltanto quattro file, tutti nel commit della pipeline di integrazione continua (27 agosto 2026) e solo per correzioni richieste dal linter: un tipo esplicito al posto di `any`, una coppia di virgolette e un apostrofo trasformati in entità HTML, un commento per il linter.

```
$ git diff --stat baseline-pre-cr-bf-01 7469969 -- src prisma
 src/app/api/admin/richieste/route.ts            | 8 +++++++-
 src/app/libri/[id]/page.tsx                     | 2 +-
 src/app/prenota/page.tsx                        | 2 +-
 src/components/accessibility/live-announcer.tsx | 5 +++++
 4 files changed, 14 insertions(+), 3 deletions(-)
```

### 2.1.3 Esito

La suite è stata eseguita sul commit `7469969`, che contiene l'infrastruttura e due dei cinque file di test, insieme ai file `prenotazioni.test.ts`, `login.test.ts` e `admin-ui.test.ts` nella versione poi integrata nei commit `4de697b` e `6f59dc4`: 5 file e 34 casi su 34 conformi all'oracolo. Nei documenti di test il riferimento di esecuzione è indicato accanto al tag, così che chiunque possa riprodurre il risultato.

## 2.2 Test Incident n.2

**Componente:** Test di Sistema – `tests/pre-modifica/automation-notifications.test.ts`

**Severità:** Minore (portabilità del setup di test)

**Data:** 4 settembre 2026

### 2.2.1 Descrizione del Problema

Durante la riesecuzione dei casi invariati come regressione, la suite delle automazioni e delle notifiche non riusciva a preparare il database. Prima di ogni esecuzione il setup sincronizza lo schema del database di test lanciando la CLI di Prisma. Su Windows il file eseguibile è `prisma.cmd` e viene lanciato attraverso la shell; la cartella di lavoro conteneva uno spazio nel percorso, la shell spezzava il percorso dell'eseguibile in due parole e il comando non veniva trovato. L'output dell'errore non è stato conservato nel report di regressione.

### 2.2.2 Risoluzione Tecnica

**Codice Errato:** il setup passa alla shell un percorso non racchiuso tra virgolette.

```
const result = spawnSync(executable, ["db", "push"], {
  cwd: process.cwd(),
  env: { ...process.env, DATABASE_URL: databaseUrl },
  encoding: "utf8",
```

```
  shell: process.platform === "win32",
});
```

Il difetto è nel codice di preparazione del test; l'ambiente, cioè una cartella con uno spazio nel nome, lo rende visibile. Il sistema sotto test non è coinvolto. Per non alterare né i test né l'applicazione durante una verifica di regressione, la stessa copia di lavoro è stata esposta tramite un collegamento (junction) a un percorso senza spazi e i cinque casi interessati sono stati rieseguiti da lì, filtrandoli per identificativo:

```
npx vitest run tests/pre-modifica/automation-notifications.test.ts \
  -t "PRE-AUT-00[124]|PRE-NOT-00[12]"
```

### 2.2.3 Esito

I cinque casi sono risultati conformi alla baseline. L'incidente è stato classificato come difetto di portabilità del setup di test, aggirato senza modificarlo. Su Linux la chiamata non usa la shell; la pipeline di integrazione continua, d'altra parte, dal 2 settembre 2026 non esegue la suite di caratterizzazione (`npm test` esclude `tests/pre-modifica`).

## 2.3 Test Incident n.3

**Componente:** Test di Sistema – `tests/pre-modifica/automation-notifications.test.ts`

**Severità:** Maggiore (suite non eseguita)

**Data:** 12 settembre 2026

### 2.3.1 Descrizione del Problema

Durante la correzione del middleware, che da quel momento verifica davvero il token di sessione tramite Auth.js, è emerso che l'intera suite delle automazioni e delle notifiche non veniva più eseguita da tempo. Il modulo `src/lib/auth.ts`, importato a cascata dalle route sotto test, carica `next-auth`; questo importa `next/server`, e il resolver ESM di Node non riesce a risolverlo perché il pacchetto di Next non dichiara il campo `exports`:

```
Error: Cannot find module '.../node_modules/next/server'
Did you mean to import "next/server.js"?
```

L'errore avveniva nel `beforeAll`, all'import dinamico della route: Vitest segnava i sei casi come *skipped* e il file come suite fallita. Nessuno se ne era accorto perché dal 2 settembre 2026 `npm test`, e quindi la pipeline di integrazione continua, esclude la cartella `tests/pre-modifica`: la suite di caratterizzazione si esegue solo con `npm run test:pre`.

### 2.3.2 Risoluzione Tecnica

**Codice Errato:** la configurazione lasciava a Node il caricamento dei pacchetti di Auth.js.

```
test: {
  environment: "node",
  include: ["tests/**/*.test.ts", "tests/**/*.spec.ts"],
```

```
  passWithNoTests: true,
  ...
},
```

**Codice Corretto:** i due pacchetti passano dalla trasformazione di Vite, che risolve correttamente l'estensione del modulo. Non cambia ciò che viene testato, solo come viene caricato.

```
test: {
  environment: "node",
  ...
  server: {
    deps: {
      inline: [/next-auth/, /@auth\/core/],
    },
  },
},
```

Una volta caricata, la suite ha rivelato che il caso PRE-NOT-002 chiamava l'elenco delle notifiche senza sessione: nel frattempo la route era stata corretta di proposito per usare sempre l'utente autenticato invece di un parametro `userId` preso dall'indirizzo. Al test è stato aggiunto il mock della sessione, così che continui a verificare ciò che gli interessa: filtro, paginazione e conteggi.

### 2.3.3 Esito

La suite torna a essere caricata ed eseguita con `npm run test:pre`. L'incidente ha mostrato che una suite esclusa dal comando di routine non segnala i propri errori: la suite di caratterizzazione va rieseguita con `npm run test:pre` quando cambiano i moduli che importa, controllando anche il numero di casi effettivamente eseguiti.

## 2.4 Test Incident n.4

**Componente:** Test di Sistema – `TC-PRE-016` (`tests/pre-modifica/admin-ui.test.ts`)

**Severità:** Minore

**Data:** 12 settembre 2026

### 2.4.1 Descrizione del Problema

Il caso TC-PRE-016 era stato scritto per fotografare il comportamento della baseline: con un cookie di sessione presente, il middleware lascia proseguire la richiesta verso `/prenota`. Il test usava però un valore *inventato* (`token-baseline`) e si aspettava che passasse.

Quando il middleware è stato corretto per verificare firma e scadenza del token, il test è fallito: la verifica del token è asincrona e un valore inventato porta ora al login, mentre il test chiamava il middleware in modo sincrono e pretendeva che la richiesta proseguisse.

Il fallimento non indicava una regressione. Il test non descriveva un requisito ma il difetto stesso: in produzione bastava inviare un cookie qualsiasi (`authjs.session-token=x`)

per ottenere dati reali. Un test scritto così avrebbe fatto apparire la correzione come una regressione.

### 2.4.2 Risoluzione Tecnica

**Codice Errato:** il test accettava un cookie inventato e verificava una chiamata sincrona.

```
it("[TC-PRE-016] lascia proseguire la UI prenotazione con cookie di sessione", ()
    => {
  const response = appMiddleware.middleware(
    request("http://localhost/prenota", "GET", undefined,
      "authjs.session-token=token-baseline"));
  expect(response.status).toBe(200);
  expect(response.headers.get("x-middleware-next")).toBe("1");
});
```

**Codice Corretto:** il caso è stato capovolto e reso asincrono, come la verifica reale del token. Il caso complementare con un token firmato davvero, senza il quale anche un middleware che rifiuta tutti sembrerebbe corretto, è stato aggiunto nella stessa modifica fra i test di unità del middleware (`tests/unit/middleware-coda.test.ts`).

```
it("[TC-PRE-016] un cookie di sessione inventato non apre piu' la UI prenotazione",
     async () => {
  const response = await appMiddleware.middleware(
    request("http://localhost/prenota", "GET", undefined,
      "authjs.session-token=token-baseline"));
  expect(response.status).toBe(307);
  const destinazione = new URL(response.headers.get("location") as string);
  expect(destinazione.pathname).toBe("/login");
});
```

### 2.4.3 Esito

Il caso è classificato come **modificato** rispetto alla baseline, con la motivazione documentata nel test stesso. La correzione ha confermato che i test di caratterizzazione vanno riletti quando il difetto che fotografano viene corretto: altrimenti diventano un vincolo che protegge il difetto.